Tablero de tareas

1. Qué vas a construir

Un tablero de tareas de clase con tres columnas («Por hacer», «En curso» y «Hecha»). La página ya tiene su HTML y su CSS: tu trabajo es programar el JavaScript para que:

- se valide el formulario de «Nueva tarea» con mensajes propios y una regla que limita las horas pendientes;
- se calculen datos a partir del array de tareas (búsqueda, grupos por estado y horas por asignatura);
- se pinten las tarjetas en su columna y un resumen de horas y progreso;
- se puedan mover las tarjetas de una columna a otra, añadir tareas nuevas y archivar las hechas.

Los apartados 1 y 2 no necesitan que se vea nada en el tablero, así que se pueden probar solos; el 3 usa funciones del 2, y el 4 junta todo.

2. Archivos entregados

Archivo	Qué es	¿Lo tocas?
`index.html`	Página con el formulario, el buscador, el resumen y las	No
`estilos.css`	Aspecto del tablero	No
`js/datos.js`	Exporta `tareasIniciales`, el array con las 9 tareas de	No
`js/principal.js`	Constantes, elementos ya seleccionados y las funciones	**Sí**

Cada tarea de `tareasIniciales` es un objeto con esta forma:

```
{ id: 2, titulo: 'Maquetar el formulario de contacto', asignatura: 'DIW',
  horas: 4, estado: 'encurso', prioridad: 'alta' }
// estado:    'porhacer' | 'encurso' | 'hecha'
// prioridad: 'baja' | 'media' | 'alta'
```

En total hay 31 h de trabajo, de las cuales 25 h están pendientes (las tareas que no están hechas), que hay que repartir entre todas las tareas.

3. Cómo trabajar

1. Escribe tu código solo en `js/principal.js`, dentro de cada función y en los huecos marcados. No cambies el nombre ni los parámetros de las funciones.
2. Las funciones del apartado 2 no tocan la página, así que puedes probarlas en cuanto las escribas con `console.log` al final del archivo:

```
console.log(buscarTareas(tareasIniciales, ' dwes '));
console.log(agruparPorEstado(tareasIniciales));
console.log(horasPorAsignatura(tareasIniciales));
```

4. Normas

- No modifiques `index.html`, `estilos.css` ni `datos.js`.
- Crea los elementos con `createElement` y asigna el texto con `textContent`. No uses `innerHTML` para pintar.
- Los datos originales (`tareasIniciales`) no se modifican: trabaja siempre con el array `tareas`.

Apartado 1 · Formulario y validación (3,0 puntos)

El navegador sabe validar formularios, pero aquí esa validación está desactivada (`noValidate`): leerás tú el estado de cada campo con la API de validación y escribirás los mensajes. Cada control va seguido de un `<small class="error">` donde se muestra su mensaje.

```
<div class="campo">
  <label for="horas">Horas estimadas</label>
  <input type="number" id="horas" name="horas" required min="1" max="20" step="any">
  <small class="error"></small>
</div>
```

Los campos son: `titulo` (obligatorio, mínimo 3 caracteres), `asignatura` (obligatorio), `horas` (obligatorio, de 1 a 20) y `prioridad` (siempre tiene valor).

1.1 · `mensajeDeError(campo)`

Devuelve el texto del primer problema que tenga el campo, o una cadena vacía si es válido. Mira `campo.validity` y respeta este orden de prioridad:

Si el campo tiene…	Devuelve
`valueMissing`	`Este campo es obligatorio`
`tooShort`	`Escribe al menos 3 caracteres` (el 3 sale del propio
`rangeUnderflow` o `rangeOverflow`	`Debe estar entre 1 y 20` (los límites salen del propio
`customError`	el `validationMessage` del campo
ninguno de ellos	`''`

1.2 · `validarCampo(campo)`

Calcula el mensaje con `mensajeDeError`, lo escribe en el elemento que va justo después del campo y marca o desmarca el campo con la clase `invalido` según tenga o no error. Devuelve `true` si el campo es válido y `false` si no.

1.3 · `comprobarHoras()`

Regla: las horas pendientes (las de las tareas que **no** están hechas) más las horas que se están escribiendo en el formulario no pueden superar `LIMITE_HORAS`. Si se superan, el campo `#horas` recibe un error personalizado; si no, se le quita cualquier error personalizado.

- Mensaje: `Superas el límite de 40 h pendientes (ahora hay 25 h)`, donde 25 son las horas pendientes actuales, sin contar lo que se ha escrito.
- Si el campo está vacío cuenta como 0 horas.
- Llegar justo al límite (40) es válido; pasarse, no.

Con los datos de partida (25 h pendientes):

Escribes en horas	Resultado
15	válido (25 + 15 = 40)
16	error: 41 h superan el límite
20	error

1.4 · `limpiarFormulario()`

Deja el formulario como recién cargado: valores iniciales, ningún mensaje de error, ningún campo con la clase `invalido` y sin error personalizado en `#horas`.

1.5 · Eventos del formulario

Usa **un solo listener por evento**, puesto en el formulario y aprovechando `evento.target` (delegación), no uno por cada campo:

- `change` : llama a `comprobarHoras()` y valida el campo que cambió.
- `input` : solo si el campo tiene la clase `invalido`, repite la comprobación de las horas y revalida ese campo. Así el error desaparece en cuanto se corrige, pero no se avisa de errores mientras el usuario escribe por primera vez.
- `keydown` : si la tecla es `Escape`, limpia el formulario.

Apartado 2 · Cálculos con los datos

Tres funciones: reciben la lista por parámetro, no tocan la página ni la variable `tareas` y devuelven un resultado nuevo. Son la base del apartado 3, así que merece la pena probarlas bien antes de seguir.

2.1 · `buscarTareas(lista, texto)`

Devuelve las tareas cuyo título **o** cuya asignatura contienen el texto, sin distinguir mayúsculas de minúsculas y sin tener en cuenta los espacios al principio y al final del texto. Con el texto vacío devuelve todas las tareas (en un array nuevo).

```
buscarTareas(tareasIniciales, ' dwes ');   // las tareas 6 y 9
buscarTareas(tareasIniciales, 'FORMULARIO'); // las tareas 2 y 7
buscarTareas(tareasIniciales, '');         // las 9 tareas
```

2.2 · `agruparPorEstado(lista)`

Devuelve un objeto con **siempre** tres propiedades (`porhacer`, `encurso` y `hecha`), cada una con un array de las tareas de ese estado en el mismo orden que tenían en `lista`. Si un estado no tiene tareas, su array queda vacío.

```
agruparPorEstado(tareasIniciales);
// { porhacer: [tareas 3, 6, 7, 9], encurso: [tareas 2, 4], hecha: [tareas 1, 5, 8] }
```

2.3 · `horasPorAsignatura(lista)`

Suma las horas de cada asignatura y devuelve un array de objetos `{ asignatura, horas }`, ordenado de más a menos horas. Si dos asignaturas tienen las mismas horas, van por orden alfabético. Con una lista vacía devuelve `[]`.

```
horasPorAsignatura(tareasIniciales);
// [ { asignatura: 'DWEC', horas: 9 },
//   { asignatura: 'DWES', horas: 9 },
//   { asignatura: 'BBDD', horas: 8 },
//   { asignatura: 'DIW',  horas: 5 } ]
```

Apartado 3 · Pintar la página

3.1 · `crearTarjeta(tarea)`

Devuelve (sin insertarla en la página) la tarjeta de una tarea. Para la tarea 2 debe quedar equivalente a este HTML:

```
<li class="tarjeta prioridad-alta" data-id="2">
  <strong class="titulo">Maquetar el formulario de contacto</strong>
  <span class="meta">DIW · 4 h</span>
  <button type="button" data-accion="retroceder">←</button>
  <button type="button" data-accion="avanzar">→</button>
</li>
```

- La clase de prioridad depende de la tarea: `prioridad-baja`, `prioridad-media` o `prioridad-alta`.
- El botón de retroceder está desactivado (`disabled`) si la tarea está en la primera columna, y el de avanzar si está en la última. Compara con `ESTADOS` en lugar de escribir los nombres de memoria.
- Todo se construye con `createElement`, `textContent`, `dataset` y `classList`/`className`; nada de `innerHTML`.

3.2 · `pintarTablero(lista)`

Vacía las tres columnas y coloca en cada una las tarjetas de las tareas de `lista` que le corresponden por estado (usa `agruparPorEstado` y `crearTarjeta`). Después escribe en cada contador (`#num-porhacer`, `#num-encurso` y `#num-hecha`) cuántas tarjetas tiene su columna.

3.3 · `pintarResumen(lista)`

Rellena el panel de resumen:

- En `#horas-asignatura`, un `<li>` por asignatura, en el orden de `horasPorAsignatura`, con el formato `DWES — 9 h` (guion largo, con espacios).
- En `#progreso`, el texto `Hechas: 3 de 9 (33 %)`: tareas hechas, tareas totales y porcentaje redondeado a un entero. Con una lista vacía debe quedar `Hechas: 0 de 0 (0 %)` (cuidado con dividir entre cero).

> ***Comprobación:*** *mientras no hagas el apartado 4, añade temporalmente `repintar();` al final del archivo. Deberías ver 4 tarjetas en «Por hacer», 2 en «En curso» y 3 en «Hecha», y en el resumen `DWEC — 9 h`, `DWES — 9 h`, `BBDD — 8 h`, `DIW — 5 h` y `Hechas: 3 de 9 (33 %)`.*

Apartado 4 · Interacción

Aquí se cambian los datos y se vuelve a pintar. Recuerda: **primero se modifica el array `tareas` y después se llama a `repintar()`**; las funciones de datos no tocan la página.

4.1 · `moverTarea(id, paso)`

Busca en `tareas` la tarea con ese `id` y cambia su estado `paso` posiciones dentro de `ESTADOS` (`+1` la lleva a la columna siguiente y `-1` a la anterior). Si no existe ninguna tarea con ese `id`, o si el movimiento la sacaría de las tres columnas, no hace nada y no da error.

```
moverTarea(3, 1);   // la tarea 3 pasa de 'porhacer' a 'encurso'
moverTarea(3, -1);  // y vuelve a 'porhacer'
moverTarea(3, -1);  // no hace nada: ya está en la primera columna
moverTarea(99, 1);  // no hace nada: no existe
```

4.2 · `anadirTarea(datos)`

`datos` trae `titulo`, `asignatura`, `horas` y `prioridad`. Añade a `tareas` una tarea con esos valores, con `estado: 'porhacer'` y con un `id` igual al mayor `id` existente más uno (1 si no hubiera tareas). Devuelve la tarea creada.

4.3 · `archivarHechas()`

Elimina de `tareas` todas las tareas hechas y devuelve cuántas eran. Si no hay ninguna hecha no cambia nada y devuelve 0.

4.4 · Eventos del tablero y arranque

- Un único `click` en `#tablero` que, con `evento.target.closest('button[data-accion]')`, detecta si se pulsó uno de los botones de las tarjetas. Si es así, obtiene el `id` de su tarjeta (ojo: `dataset` devuelve texto), mueve la tarea `+1` si la acción es `avanzar` o `-1` si es `retroceder`, y repinta.
- `input` del buscador: repinta.
- `click` de `#boton-archivar`: archiva las hechas y repinta.
- Primer pintado: llama a `repintar()` al arrancar la página.

4.5 · Envío del formulario

Al enviar el formulario:

1. Se evita el envío real de la página.
2. Se comprueba el límite de horas.
3. Se validan **todos** los campos y se muestran **todos** los errores a la vez.
4. Si hay algún error, el foco va al primer campo inválido y no se hace nada más.
5. Si todo es correcto, se añade la tarea (con el título sin espacios sobrantes y las horas como número), se limpia el formulario y se repinta.
6. En `#mensaje-formulario` aparece `Tarea añadida: TÍTULO`, con el título ya sin espacios sobrantes.